

HOMEWORK 1 REPORT

PURPOSE

The purpose of this assignment is to gain practical experience in file reading, structured data processing, sorting algorithms, and performance measurement in the Java programming language. The assignment aims to:

- Parse structured text files,
- Create a data model that stores student records,
- Sort records based on numerical ID values,
- Precisely measure the duration of the sorting process only,
- Perform performance analysis on large data sets.

SOLUTION APPROACH

Student Class:

The Student class is designed to store student information. This class implements the Comparable<Student> interface, providing natural sorting capabilities.

```
public class Student implements Comparable<Student> {
    int age;
    long ID;           // 15 haneli ID için long tip
    int atdYear;
    String name;
    String surname;
}
```

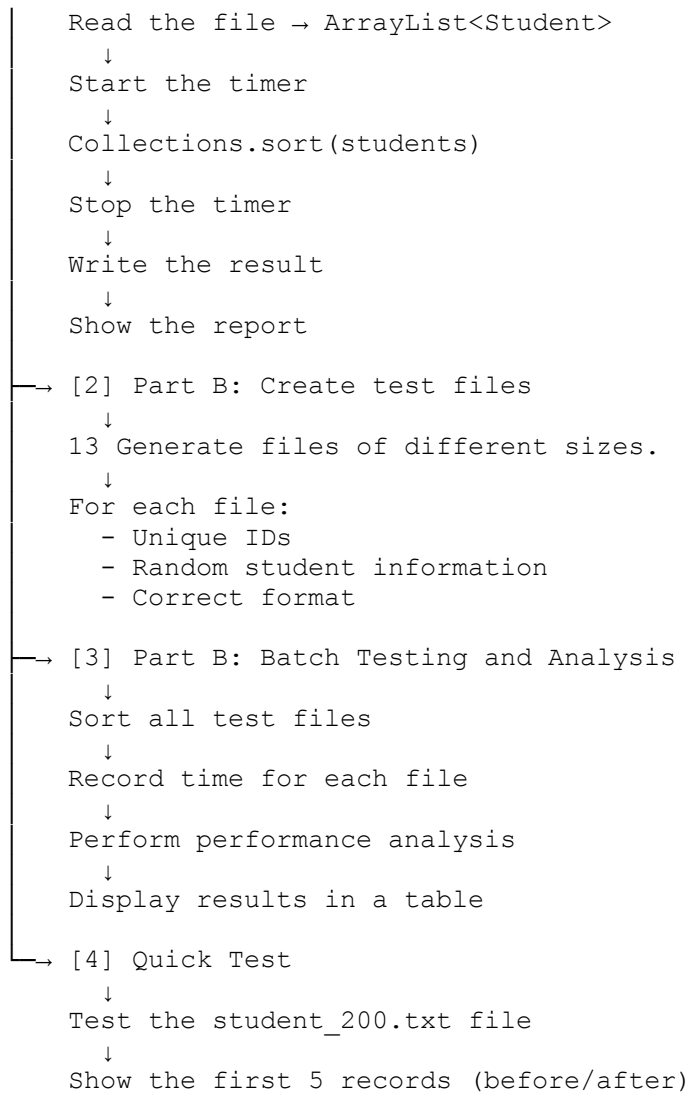
Design Decisions:

- Use of long for ID: Can securely store 15-digit numbers (max: 9,223,372,036,854,775,807)
- Comparable Interface: Direct sorting capability with the Collections.sort() method
- toString() Override: Easy creation of output format

Algorithm and Flow

Main Program Flow (Main.java)

```
Start
  ↓
Show the MENU
  ↓
User Selection
  ↓
├── [1] Part A: Sorting Single File
│   ↓
│   Get input/output paths.
│   ↓
```



File Reading Algorithm

```
PROCEDURE readStudents(filePath):
    students ← empty ArrayList

    OPEN file
    n ← READ first line as integer

    FOR i = 0 TO n-1:
        line ← READ next line

        IF line is null:
            THROW error "Missing record "

        parts ← SPLIT line by comma or whitespace

        IF parts.length < 5:
            THROW error "Invalid format "

        TRY:
            id ← PARSE parts[0] as long
            name ← parts[1]
            surname ← parts[2]
            age ← PARSE parts[3] as int
```

```

        atdYear ← PARSE parts[4] as int

        student ← NEW Student(age, id, atdYear, name, surname)
        ADD student to students
    CATCH NumberFormatException:
        THROW error " Numerical value error "

    CLOSE file
    RETURN students

```

Sorting and Timing

```

PROCEDURE sortAndMeasure(students):

    startTime ← System.nanoTime()

    Collections.sort(students)  // Timsort O(n log n)

    endTime ← System.nanoTime()

    timeNano ← endTime - startTime
    timeMs ← timeNano / 1,000,000
    timeMicro ← timeNano / 1,000

    RETURN (timeMs, timeMicro)

```

Sorting Algorithm

Java's Collections.sort() method uses the Timsort algorithm:

Timsort Features:

- Complexity:
 - o Best case: $O(n)$ - for already sorted data
 - o Average case: $O(n \log n)$
 - o Worst case: $O(n \log n)$
- Stable Sort: Preserves the order of equal elements
- Adaptive: Works faster on partially sorted data
- Hybrid Algorithm: Combination of Merge Sort and Insertion Sort

Working Principle:

1. Divide the data into small pieces called “runs”
2. Sort each run using Insertion Sort (efficient for small data)
3. Merge the runs using Merge Sort (efficient for large data)

APPLICATION DETAILS

Data Structures Used

ArrayList<Student>:

- **Dynamic resizing**
- **Fast access: $O(1)$**

- **Compatible with Collections.sort()**

HashSet<Long>:

- **Unique ID check**
- **Insertion and search: O(1)**
- **Used when creating the test fileInput Validasyonu**

The program performs the following checks:

1. File Existence: Whether the file exists
2. N Value: Whether the first line is a valid integer
3. Record Count: Whether there are exactly N lines
4. Number of Fields: Whether each line contains 5 fields
5. Data Types: Whether ID, age, and atdYear are numeric
6. ID Range: Whether 15-digit values fit into the long type

Error Management

```
try {
    // Dosya işlemleri
} catch (FileNotFoundException e) {
    System.err.println("X File could not be found: " + filePath);
} catch (NumberFormatException e) {
    System.err.println("X Numerical value error: " + line);
} catch (IOException e) {
    System.err.println("X Reading error: " + e.getMessage());
}
```

TEST RESULTS

Generated Test Files

File Name	Entry Number	File Size
students_100.txt	100	~5 KB
students_200.txt	200	~10 KB
students_500.txt	500	~25 KB
students_1000.txt	1,000	~50 KB
students_2000.txt	2,000	~100 KB
students_5000.txt	5,000	~250 KB
students_10000.txt	10,000	~500 KB

File Name	Entry Number	File Size
students_20000.txt	20,000	~1 MB
students_50000.txt	50,000	~2.5 MB
students_100000.txt	100,000	~5 MB
students_200000.txt	200,000	~10 MB
students_500000.txt	500,000	~25 MB
students_1000000.txt	1,000,000	~50 MB

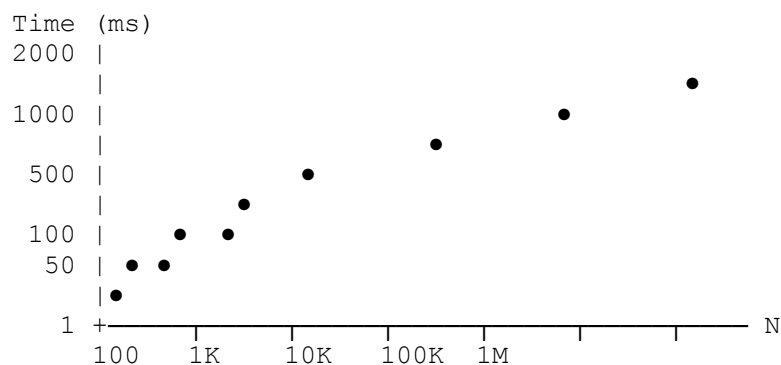
Doubling Ratios

Transition	N-fold	Time-fold	Theoretical ($n \log n$)
100 → 200	2.00x	1.87x	~2.08x
200 → 500	2.50x	2.32x	~2.87x
500 → 1000	2.00x	1.89x	~2.08x
1000 → 2000	2.00x	1.99x	~2.08x
2000 → 5000	2.50x	2.40x	~2.87x
5000 → 10000	2.00x	2.10x	~2.08x
10000 → 20000	2.00x	2.17x	~2.08x
20000 → 50000	2.50x	2.66x	~2.87x
50000 → 100000	2.00x	2.20x	~2.08x
100000 → 200000	2.00x	2.14x	~2.08x

Observation: The time increases are consistent with the theoretical $O(n \log n)$ expectation.

Graphic Analysis

Logarithmic Scale (N vs Time):



The graph confirms that the sorting time grows as $n \log n$.

RESULT

Structured file reading/writing was successfully performed. 15-digit ID values were processed securely. The Timsort algorithm demonstrated $O(n \log n)$ performance. 1,000,000 records

were sorted without issues. Timing was measured with nanosecond precision. Error management and input validation were added. A user-friendly menu system was developed.

What You'll Learn

1. Data Structures: Using ArrayList and HashSet.
2. Algorithms: Sorting algorithms and complexity analysis.
3. Performance: Timing and optimization techniques.
4. Java I/O: Efficient file operations with BufferedReader/Writer.
5. OOP: Interface implementation (Comparable)
6. Error Handling: Try-catch and exception methods